

LAB MANUAL

# The Bookshelf on Agent Search — console edition

*Day 1 • one lab in six parts, on Google Cloud's managed pieces, from the console*

Version 1.0 • September 2026

Environment: Cloud Shell in your sandbox GCP project • Agent Search (Discovery Engine API) • the Google Cloud console • Cloud Shell for Part F only

# Contents

How to use this manual

Lab 1 — The Bookshelf on Agent Search — console edition

Part A — Set-up: the bucket, the corpus, the APIs

Part B — Data store: create it, watch the import, make the fields filterable

Part C — The app on the data store

Part D-1 — The day's question, no filter

Part D-2 — The same search, three filters

Part D-3 — The answer endpoint, as daniel and as priya

Part E — The console preview pane

Part F — Measuring: forty questions, the judge, the thresholds

Teardown

Appendix A — The golden set

Appendix B — The callers and their filter strings

Appendix C — eval.yaml, verbatim

# How to use this manual

This is **one lab in six parts**, A to F, run one part at a time right after the chapter of the deck that explains it. The whole retrieval pipeline is built from Google Cloud's managed pieces: a **data store** that parses, chunks, embeds and indexes the documents; a **search app** on top of it that retrieves, reranks and writes grounded answers with citations; and the **Vertex AI evaluation service** that judges those answers. In this **console edition** you click through the console for Parts A, B, C and E; in Cloud Shell you run the kit's scripts — `00-upload.sh` uploads the corpus in Part A, `search.sh` and `answer.sh` send the questions in Part D, and `evaluate.py` runs the forty questions in Part F. You write no code.

Google has renamed Vertex AI Search to **Agent Search**, and Vertex AI as a whole now sits under the Gemini Enterprise Agent Platform. The API underneath did not change: it is the **Discovery Engine API** ([discoveryengine.googleapis.com](https://discoveryengine.googleapis.com)). The console still says "AI Applications" in places, and the newest documentation pages say "Gemini Enterprise". This manual says Agent Search and means all of that.

## Console labels change

Google renames console pages, tabs and buttons often. Every console step in this manual names the control as Google's documentation names it **and** the API field it sets, so if a label on your screen differs you can still find the right control by what it does. If a page has moved, ask in the room.

The same kit is used by a **script edition** of this manual, in which `scripts/*.sh` make every API call from Cloud Shell. The two editions build the **same thing**: the same bucket name, the same data store with the same parser, chunk size and headings flag, the same schema flags, the same app tier and add-on. Where the console cannot set something the scripts set, this edition says so in a warning box and names the script to run instead.

Blocks captioned **expected output** show what a console page or an API response looks like at that step; values that depend on your project — ids, timings, ranks, scores — are shown as `<n>`, ... or `my-project`. Blocks captioned **example** show one complete response so you can see every field before you read your own. Blocks captioned **kit file** — **local** print a file from the kit.

## The story for today

Meridian Financial keeps its policies, procedures and product handbooks in a document library called the Bookshelf. Today the Bookshelf is 13 documents — approved, draft and superseded; classified internal, confidential and restricted — and the question is whether staff can ask it questions and get an answer that is correct, cited, and only from documents they are allowed to read.

Three staff profiles are used throughout, from `callers.json` (printed in Appendix B): **priya** (Priya Nair, Customer Contact agent, cleared for `internal`), **daniel** (Daniel Okafor, Dispute Officer, cleared for `confidential`) and **mlro** (Sarah Whitmore, Deputy MLRO, cleared for `restricted`). Each one has a filter string; in Part D you paste it into the request; in Part F `evaluate.py` sends it on every request; the callers never type it.

One question runs through the day: **"Which form must a cardholder sign before a dispute can go to chargeback?"** You search it with no filter in Part D-1, with three filters in Part D-2, and ask the answer endpoint as two callers in Part D-3. Part F then asks forty questions and counts.

**Cost.** The search app is created with the Enterprise tier and the LLM add-on. Both are required for generated answers and both are charged for. `evaluate.py` sends 40 answer requests per run and then calls the evaluation service. Do the teardown at the end of the day.

## Set-up (once, before Part A)

Open the Google Cloud console, select your sandbox project in the project picker at the top, and write down its **project id** (not its name, not its number): every resource string in Part D starts with it, and it goes into `lab.env` now.

Open Cloud Shell in the same project. Upload `regai-d1-gcp-kit.zip` (the upload button in the Cloud Shell toolbar), then:

### KIT FILE — LOCAL. THE LISTING, IN COLUMNS; YOUR TERMINAL WIDTH DECIDES THE LAYOUT

```
$ unzip regai-d1-gcp-kit.zip && cd regai-d1-gcp-kit
$ ls
README.md      callers.json    datastore.json  golden.json     schema.json
app.json        corpus-html     eval.yaml       lab.env.example scripts
corpus-md       fixtures        metadata.jsonl
```

`lab.env` is the one kit file you edit today. Copy the example and set `PROJECT_ID` to your project **id**. `BUCKET` is computed from it as `<project-id>-regai-corpus` — the name you give the bucket in Part A step 2. Leave `LOCATION=global`; the defaults for the rest are fine until Part F, when `DATASTORE_ID` and `APP_ID` get the ids the console shows.

### KIT FILE — LOCAL, WITH `PROJECT_ID` FILLED IN. EVERY KIT SCRIPT SOURCES THIS AND REFUSES TO RUN WHILE `PROJECT_ID` IS EMPTY

```
$ cp lab.env.example lab.env
$ nano lab.env           # or the Cloud Shell editor: set PROJECT_ID=
$ grep -v '^#' lab.env | grep .
PROJECT_ID=my-project
LOCATION=global
BUCKET=${PROJECT_ID}-regai-corpus
BUCKET_LOCATION=europe-west2
DATASTORE_ID=bookshelf-ds
APP_ID=bookshelf-app
ANSWER_MODEL=stable
VERTEX_LOCATION=europe-west2
```

The Cloud Shell **Editor** (the **Open Editor** button) is where you read kit files during the day: `metadata.jsonl` in Part A, two `corpus-html/*`.html files in Part D-1, one line of `scripts/answer.sh` in Part D-3. Keep the Cloud Shell tab open beside the console tab.

The whole day, in order — every step is explained in its part:

### THE ORDER — CONSOLE EDITION

```
Part A  Cloud Storage → Buckets → Create; bash scripts/00-upload.sh (corpus-html/ and
AI      metadata.jsonl into the bucket); APIs & Services → enable Discovery Engine, Vertex
Part B  AI Applications → Data Stores → Create data store (Cloud Storage, JSONL with
        metadata, global, bookshelf-ds, layout parser, 250 tokens, ancestor headings);
        Activity tab; Schema tab (status, classification, department indexable; title)
Part C  AI Applications → Create app (Enterprise on, Generative Responses on, bookshelf-
app,    global, attach bookshelf-ds); Preview once for readiness
Part D  Cloud Shell: bash scripts/search.sh and scripts/answer.sh
        D-1 no filter · D-2 three filters · D-3 answer as daniel and as priya
Part E  the app's Preview page, the day's question
Part F  Cloud Shell: python scripts/evaluate.py, then --no-status-filter
Teardown AI Applications: delete the app, then the data store; Cloud Storage: the bucket
```

## LAB 1 • DIFFICULTY ●●●

# Lab 1 — The Bookshelf on Agent Search — console edition

## Objective

Build the Bookshelf's question-answering on Google Cloud's managed pieces, from the console, and prove, with numbers, that it answers from approved documents only, to the caller who may see them. By the end of Part F you have: a bucket with the corpus; a data store that parsed and chunked it with the layout parser; a search app with the Enterprise tier and the LLM add-on; one question searched with and without filters and answered for two callers, from Cloud Shell; the console's preview of the same answer; and forty golden questions scored locally and by the Vertex AI evaluation service, with thresholds that can fail — and a weakened run that shows them failing.

## Environment

Your browser, signed in to the Google Cloud console of your own sandbox project, plus Cloud Shell in the same project with the kit `regai-d1-gcp-kit.zip` unzipped and `lab.env` filled in. Parts A to C, E and the teardown are console pages; Part D and Part F run kit scripts in Cloud Shell, and `scripts/00-upload.sh` does the upload in Part A; `python scripts/evaluate.py` in Part F.

## Prerequisites

- A sandbox project with billing enabled, and your account holding **Discovery Engine Admin** (or Owner), **Storage Admin** and **Vertex AI User**
- The set-up above done: the console open on your project, the kit unzipped in Cloud Shell, `lab.env` copied and `PROJECT_ID` set
- The deck's chapter for each part taught before you start that part — each part assumes its chapter

## The six parts

**A — Set-up.** Create the bucket in the console, upload the 13 HTML files and `metadata.jsonl` with `00-upload.sh`, enable the two APIs in the console.

**B — Data store.** Create it from the console with the layout parser and 250-token chunks; watch the import on the Activity tab; mark the metadata fields filterable on the Schema tab.

**C — App.** Create the search app (Enterprise tier, LLM add-on) on the data store; one preview query to see the index is ready.

**D — Search, filter, answer.** From Cloud Shell with two kit scripts. D-1: the day's question, no filter. D-2: the same search with three filters. D-3: the answer endpoint as daniel and as priya.

**E — The console preview.** The same question in the app's Preview page, citations highlighted.

**F — Measuring.** Forty questions, local metrics, the evaluation service, thresholds — then the weakened run. Then teardown.

## Part A — Set-up: the bucket, the corpus, the APIs

### Goal

Get the corpus where the data store can read it: create the bucket in the console, upload `corpus-html/` and `metadata.jsonl` into it with the kit's upload script (which rewrites the placeholder prefix `gs://BUCKET/` to your bucket on the way), enable the two APIs the day needs, and look once at a document the way the parser will see it.

### Steps

1. Read the one file that matters in this part: a line of `metadata.jsonl`. Open it in the Cloud Shell Editor (or run `head -n 1 metadata.jsonl`). One document per line — an `id`, the `structData` the filters will read, and a `content` pointer whose `uri` still carries the placeholder `gs://BUCKET/`. Step 3 rewrites that placeholder to your bucket.

**KIT FILE — LOCAL. THE FIRST LINE, LAID OUT OVER FIVE LINES HERE; IN THE FILE IT IS ONE LINE**

```
{
  "id": "HB-BLB-001",
  "structData": {
    "title": "Buy Local Bonds — Product Handbook",
    "status": "approved",
    "classification": "restricted",
    "department": "Treasury Products",
    "version": "1.2",
    "effective_date": "2026-06-01"
  },
  "content": {
    "mimeType": "text/html",
    "uri": "gs://BUCKET/corpus-html/HB-BLB-001.html"
  }
}
```

2. Create the bucket. In the console open **Cloud Storage** → **Buckets** and click **Create**. Name it `<project-id>-regai-corpus` — bucket names are global, so your project id keeps it unique — which is exactly the name `BUCKET` in `lab.env` computes. Location type **Region**, region `europe-west2`; leave **Uniform** access control and **Enforce public access prevention** on; defaults for everything else. Click **Create** and write the name down.

**EXPECTED OUTPUT. THE BUCKET LIST AFTER CREATE**

| Buckets                 |              |               |                       |
|-------------------------|--------------|---------------|-----------------------|
| Name                    | Location     | Location type | Default storage class |
| my-project-regai-corpus | europe-west2 | Region        | Standard              |

3. Upload the corpus with the kit's upload script. In Cloud Shell, from the kit folder, run `bash scripts/00-upload.sh`. It checks that the bucket from step 2 exists, rewrites every `gs://BUCKET/` in a temporary copy of `metadata.jsonl` to your bucket name, uploads `corpus-html/` and that copy, and lists what is in the bucket. Each command is printed with a `+` in front before it runs.

**EXPECTED OUTPUT. LONG COMMANDS ARE PRINTED ON ONE LINE; WRAPPED HERE**

```
$ bash scripts/00-upload.sh
+ gcloud config set project my-project
Updated property [core/project].
+ sed "s#gs://BUCKET/#gs://my-project-regai-corpus/#g" metadata.jsonl \
  > /tmp/tmp.../metadata.jsonl
first rewritten line:
{"id": "HB-BLB-001", "structData": {...}, "content": {"mimeType": "text/html",
  "uri": "gs://my-project-regai-corpus/corpus-html/HB-BLB-001.html"}}
+ gcloud storage cp -r /home/you/regai-d1-gcp-kit/corpus-html gs://my-project-regai-corpus/
Copying file:///.../HB-BLB-001.html to gs://my-project-regai-corpus/corpus-html/HB-BLB-001...
... (13 files)
+ gcloud storage cp /tmp/tmp.../metadata.jsonl gs://my-project-regai-corpus/metadata.jsonl
+ gcloud storage ls -r gs://my-project-regai-corpus/
gs://my-project-regai-corpus/metadata.jsonl
gs://my-project-regai-corpus/corpus-html/:
gs://my-project-regai-corpus/corpus-html/HB-BLB-001.html
... (13 .html objects)
```

Done. Next: Part A step 4 — enable the APIs in the console.

Two lines to check before you go on: the **first rewritten line** names your bucket, not `gs://BUCKET/`, and the `ls -r` shows 13 `.html` objects plus `metadata.jsonl`. Then open **Cloud Storage** → **Buckets**, click the bucket's name, and read the **Objects** tab: `metadata.jsonl` at the top level and a `corpus-html/` folder with the 13 files.

**EXPECTED OUTPUT. THE OBJECT LIST; THE SIZE SHOWN FOR METADATA.JSONL DEPENDS ON YOUR BUCKET NAME'S LENGTH**

```
Objects - gs://my-project-regai-corpus
Name      Size      Type
corpus-html/  -      Folder
metadata.jsonl  4.2 KB  application/octet-stream

Objects - gs://my-project-regai-corpus/corpus-html
HB-BLB-001.html  HB-LN-004.html  HB-SAV-002.html  POL-AML-007.html  POL-CR-014-v2.html
POL-CR-014.html  POL-DP-002.html  POL-HR-005.html  PRC-COMP-009.html  PRC-CR-030.html
PRC-IT-021.html  PRC-KYC-003.html  PRC-OPS-011.html  (13 objects)
```

4. Enable the two APIs. Open **APIs & Services** → **Enabled APIs & services**, click **+ Enable APIs and services**, search for **Discovery Engine API** and click **Enable**; then search for `aiplatform` and enable the API it finds — **Agent Platform API** (its service name is `aiplatform.googleapis.com`; it used to be listed as Vertex AI API). Discovery Engine is the Agent Search API — Parts B to E run on it; the Agent Platform API is for the evaluation service in Part F.
5. Open one document the way the parser will see it. In the Cloud Shell Editor open `corpus-html/POL-CR-014.html` (or run the two commands below). The layout parser reads structure — `h1`, `h2`, `p`, `table` — and this is why the corpus is HTML: the parser does not read Markdown.

**KIT FILE — LOCAL. LINES 7-10 OF THE FILE, CUT SHORT, AND THE HEADING COUNT**

```
$ sed -n '7,10p' corpus-html/POL-CR-014.html | cut -c1-80
<body>
<h1>Credit Card Dispute and Chargeback Handling Policy</h1>
<h2>1. Purpose and scope</h2>
<p>This policy sets out how Meridian Financial ("the Bank") receives, assesses and r
$ grep -c '<h2>' corpus-html/POL-CR-014.html
9
```

### ✓ Check yourself

The bucket exists, in `europa-west2`, with uniform access, and its name is `<project-id>-regai-corpus` — the same as `BUCKET` in `lab.env`.

`00-upload.sh` printed `first rewritten line:` followed by a uri that starts `gs://<your bucket>/corpus-html/HB-BLB-001.html`.

The bucket's **Objects** tab shows `metadata.jsonl` at the top level and 13 `.html` objects under `corpus-html/` — the same paths the rewritten uris name.

**Discovery Engine API** and **Agent Platform API** show as enabled.

You have seen that a corpus file has an `h1`, nine `h2` headings and `p` paragraphs — the structure the parser will chunk along.

## Troubleshooting

| Symptom                                                                             | Cause                                                                                       | Fix                                                                                                                                                                       |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Create</b> refuses the bucket name ("already exists" / not available)            | Bucket names are global and someone owns that name                                          | Add a suffix, e.g. <code>&lt;project-id&gt;-regai-corpus-2</code> , and set <code>BUCKET=</code> in <code>lab.env</code> to that exact name so the script uploads into it |
| <code>lab.env</code> not found                                                      | The set-up above was skipped                                                                | <code>cp lab.env.example lab.env</code> , set <code>PROJECT_ID</code> , rerun the script                                                                                  |
| <code>PROJECT_ID</code> is empty in <code>lab.env</code>                            | <code>lab.env</code> was copied but not filled in                                           | Set <code>PROJECT_ID=&lt;your project id&gt;</code> in <code>lab.env</code> and rerun                                                                                     |
| bucket <code>gs://...</code> not found                                              | Step 2 was skipped, or the name in <code>lab.env</code> differs from the bucket you created | Create it, or make <code>BUCKET</code> in <code>lab.env</code> match the console's name exactly; rerun                                                                    |
| uri rewrite failed                                                                  | <code>metadata.jsonl</code> was edited and no longer contains <code>gs://BUCKET/</code>     | Restore it from the kit zip and rerun                                                                                                                                     |
| The <b>Objects</b> tab shows fewer than 13 files, or no <code>metadata.jsonl</code> | The upload was interrupted                                                                  | Rerun <code>bash scripts/00-upload.sh</code> — it copies the objects again                                                                                                |
| <b>Enable</b> asks for billing or a permission                                      | The project has no billing account, or you are not Owner / Service Usage Admin              | Have the project owner enable the two APIs                                                                                                                                |

## Part B — Data store: create it, watch the import, make the fields filterable

### Goal

One console flow does what three kit scripts do. **Create data store** sends `datastore.json`'s fields — the layout parser, chunk mode with 250-token chunks and ancestor headings — `*and*` starts the import from `gs://<bucket>/metadata.jsonl` in the same step (`01-create-datastore.sh` + `03-import.sh`). The **Activity** tab is where you watch the long-running operation the scripts poll. The **Schema** tab is `02-patch-schema.sh`: `status`, `classification` and `department` become **indexable** (filterable) and `title` is the document title. One difference in order, which Google's documentation forces: the console lets you edit the schema only after the import has completed, so here the schema comes last and triggers a re-index. The end state is the same.

### Steps

1. Read the create body the kit's `01-create-datastore.sh` sends, so you know what you are about to click. Three things in it are yours: the parser, the chunk size, the headings flag. Everything else — `GENERIC`, `SOLUTION_TYPE_SEARCH`, `CONTENT_REQUIRED` — the console sets for you when you choose a search data store with unstructured documents.

#### KIT FILE — LOCAL

```
$ cat datastore.json          # kit file, for reading
{
  "displayName": "Bookshelf policies",
  "industryVertical": "GENERIC",
  "solutionTypes": ["SOLUTION_TYPE_SEARCH"],
  "contentConfig": "CONTENT_REQUIRED",
  "documentProcessingConfig": {
    "chunkingConfig": {
      "layoutBasedChunkingConfig": { "chunkSize": 250, "includeAncestorHeadings": true }
    },
    "defaultParsingConfig": { "layoutParsingConfig": {} }
  }
}
```

2. In the console open **AI Applications** (the console's name for Agent Search; the newest documentation calls the page **Gemini Enterprise**). If it asks you to activate the API, accept. In the navigation menu click **Data Stores**, then **Create data store**.
3. On the **Select a data source** page select **Cloud Storage**.
4. Under **Select a folder or file you want to import** choose **File**, and in the `gs://` field enter the metadata file's path — or **Browse** to it and **Select**. This is the `gcsSource.inputUri` of the import: one JSONL file, whose lines point at the 13 HTML objects.

#### THE PATH — YOUR BUCKET NAME, THEN /METADATA.JSONL

```
gs://my-project-regai-corpus/metadata.jsonl
```

5. Under **Select what kind of data you are importing** choose the option for **unstructured documents with metadata, described by a JSONL file**. Two neighbours are wrong: *\*Structured data (JSONL)\** would index the metadata lines themselves as the documents, and *\*Unstructured documents (PDF, HTML, TXT and more)\** would import the JSONL file as one document and never read the [uris](#). The right option is the one whose description mentions metadata, or JSONL *\*with\** metadata, for unstructured documents. Click **Continue**.
6. Choose a region for your data store: **global (Global)**. `LOCATION=global` in `lab.env` and every resource string in Part D assume it. Enter a name for your data store: `bookshelf-ds`. Watch for the `id` the console derives from the name and shows next to it — it may be `bookshelf-ds` or `bookshelf-ds` with a numeric suffix. Write the `id` down: it is `DATASTORE_ID` in every Part D resource string and in Part F's `lab.env`.
7. Expand **Document processing options** (Google's documentation names this section). Set three things, matching `datastore.json`: the **parser** to the **Layout parser** (not the digital or OCR parser); **document chunking** on; **chunk size** `250` (tokens; the documented range is 100–500); and the option to **include ancestor headings** in each chunk on. Then click **Create**.

**If a setting is not on the screen, do not create the store — say so and use the API**

Google's documentation confirms that the parser and chunking can be chosen when creating a data store in the console and **cannot be changed afterwards**. If the **Document processing options** section offers the layout parser but no chunk size, or no headings option, or no chunking at all, a store created there would differ from `datastore.json` (default chunk size 500, headings off) — and the difference would be visible in Part D-1's content column and invisible everywhere else.

In that case cancel, and create the store from Cloud Shell with the kit: `scripts/01-create-datastore.sh` (sends `datastore.json` as shown in step 1) then `scripts/03-import.sh` — the kit's README says how. Everything from step 9 onward in this edition then applies unchanged, because the store is the same.

**EXPECTED OUTPUT. THE LIST AFTER CREATE**

| Data Stores  |              |               |        |           |             |
|--------------|--------------|---------------|--------|-----------|-------------|
| Name         | Type         | Data source   | Region | Documents | Status      |
| bookshelf-ds | Unstructured | Cloud Storage | global | ...       | In progress |

8. Watch the import. Click `bookshelf-ds`, then the **Activity** tab. Google's documentation: "When the status column on the Activity tab changes from **In progress** to **Import completed**, the ingestion is complete." This is the operation `03-import.sh` polls. Write down the minutes from Create to Import completed — it is the number the kit's `03-import.sh` prints as `import took <seconds> s`. Then read on in the deck while it runs.

**EXPECTED OUTPUT. THE ACTIVITY TAB, BEFORE AND AFTER**

| Activity         |                    |         |          |                         |  |
|------------------|--------------------|---------|----------|-------------------------|--|
| Type             | Status             | Started | Duration | Details                 |  |
| Import documents | In progress        | hh:mm   | ...      | gs://.../metadata.jsonl |  |
|                  | → Import completed | hh:mm   | <min>    | 13 documents, 0 errors  |  |

9. Open the **Documents** tab. It should list 13 documents by id. If it shows fewer, or an error count, the `uri` in one of the metadata lines does not match an object in the bucket — see the table below.

#### EXPECTED OUTPUT. THE 13 IDS ARE THE KIT'S

| Documents     |                                                             | 13 documents |
|---------------|-------------------------------------------------------------|--------------|
| ID            | URI                                                         |              |
| HB-BLB-001    | gs://my-project-regai-corpus/corpus-html/HB-BLB-001.html    |              |
| HB-LN-004     | gs://my-project-regai-corpus/corpus-html/HB-LN-004.html     |              |
| ...           |                                                             |              |
| POL-CR-014    | gs://my-project-regai-corpus/corpus-html/POL-CR-014.html    |              |
| POL-CR-014-v2 | gs://my-project-regai-corpus/corpus-html/POL-CR-014-v2.html |              |
| ...           |                                                             | (13 rows)    |

Do not read anything into `POL-CR-014-v2` yet beyond "there are two dispute-policy ids". The `status` each one carries is in `structData`; nothing has read it, and nothing will until Part D-2.

10. Make the metadata fields filterable — the console's form of `schema.json`. Google's documentation: "Check that you have completed data ingestion. Otherwise, the schema might not be available to edit yet." So only now: click the **Schema** tab (it may be empty until the import has completed), then **Edit**. The fields of `structData` were auto-detected from the import. For each one set the annotations to match `schema.json`, then click **Save**.

| Field          | Retrievable | Indexable | Dynamic Facetable | Key properties |
|----------------|-------------|-----------|-------------------|----------------|
| title          | —           | —         | —                 | title          |
| status         | ✓           | ✓         | ✓                 | —              |
| classification | ✓           | ✓         | ✓                 | —              |
| department     | ✓           | ✓         | ✓                 | —              |
| version        | ✓           | —         | —                 | —              |
| effective_date | ✓           | —         | —                 | —              |

**Indexable** is the one that matters today: it is `"indexable": true` in `schema.json`, and it is what lets `status: ANY("approved")` work in Part D-2. **Key properties** → **title** is `"keyPropertyMapping": "title"` — it makes **title** the document title the answer endpoint cites. The other two annotations are set because `schema.json` sets them, so that the two editions match; **Searchable** and **Completable** stay off. The console's documented annotations are **Retrievable**, **Indexable**, **Dynamic Facetable**, **Searchable**, **Completable** and a **Key properties** column; leave any annotation the table does not mention at its default.

#### EXPECTED OUTPUT. THE SCHEMA AFTER SAVE, AS A TABLE

| Schema — bookshelf-ds |        | (after Save) |           |                   |                |
|-----------------------|--------|--------------|-----------|-------------------|----------------|
| Field                 | Type   | Retrievable  | Indexable | Dynamic facetable | Key properties |
| classification        | string | ✓            | ✓         | ✓                 | —              |
| department            | string | ✓            | ✓         | ✓                 | —              |
| effective_date        | string | ✓            | —         | —                 | —              |
| status                | string | ✓            | ✓         | ✓                 | —              |
| title                 | string | —            | —         | —                 | title          |
| version               | string | ✓            | —         | —                 | —              |

Google's documentation says: "Changing the schema triggers reindexing. For large data stores, reindexing can take hours." Thirteen documents is not large. The **Activity** tab should show the re-index; **write down how long it takes** — the kit's scripts patch the schema *\*before\** the import, so this wait exists only when the schema is edited afterwards, as it is here.

11. Look, do not touch. While it re-indexes, find on the data store's pages the settings you chose in step 7 — the parser, the chunk size, the headings flag — if the console shows them anywhere. If it does not, that is worth knowing too: *\*what you can no longer see\** is one of the day's three questions.

### ✓ Check yourself

The **Activity** tab showed **Import completed**, and you wrote down the minutes.

The **Documents** tab lists 13 documents, including both **POL-CR-014** and **POL-CR-014-v2**.

The **Schema** tab, after **Save**, shows **status**, **classification** and **department** with **Indexable** on, and **title** mapped to the **title** key property.

You know the data store's **id** as the console shows it, and you have written it down.

You either saw the layout parser, chunk size 250 and ancestor headings on the screen when creating the store — or you cancelled and used **01-create-datastore.sh** + **03-import.sh**.

## Troubleshooting

| Symptom                                                                                       | Cause                                                                                                                 | Fix                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Create data store</b> is missing, or the page asks you to enable an API                    | The Discovery Engine API is not enabled, or you lack Discovery Engine Admin                                           | Part A step 4; check the role on the project                                                                                                                                                                                                                                                                                              |
| <b>Activity</b> shows an error, or <b>Documents</b> shows fewer than 13                       | A <b>uri</b> in <b>metadata.jsonl</b> does not match an object in the bucket — usually a leftover <b>gs://BUCKET/</b> | Open the error details (they name an error file in the bucket); check the bucket's <b>Objects</b> tab against Part A step 4; rerun <b>bash scripts/00-upload.sh</b> if anything is missing (it re-uploads the rewritten file), then on the <b>Documents</b> tab click <b>Import data</b> → <b>Cloud Storage</b> and import the file again |
| The <b>Schema</b> tab is empty or has no <b>Edit</b>                                          | The import has not completed                                                                                          | Wait for <b>Import completed</b> ; refresh the page                                                                                                                                                                                                                                                                                       |
| A field you need is not in the schema                                                         | It was absent from every <b>structData</b> line                                                                       | Should not happen with the kit's file; if it does, re-check your edited <b>metadata.jsonl</b> against the kit's original                                                                                                                                                                                                                  |
| <b>Save</b> on the schema is rejected for <b>title</b>                                        | The key property and another annotation conflict                                                                      | Leave only <b>Key properties</b> → <b>title</b> on <b>title</b>                                                                                                                                                                                                                                                                           |
| Import completed, but Part C's preview returns nothing                                        | Indexing (or the re-index after the schema change) continues after the operation completes                            | Wait a few minutes and retry. Not an error                                                                                                                                                                                                                                                                                                |
| No <b>chunk size</b> / <b>ancestor headings</b> control in <b>Document processing options</b> | The console does not expose the setting                                                                               | The warning box above: cancel; <b>01-create-datastore.sh</b> + <b>03-import.sh</b> from Cloud Shell                                                                                                                                                                                                                                       |

## Part C — The app on the data store

### Goal

**Create app** is `04-create-app.sh`: a search app (the API calls it an engine) with the two toggles that are `SEARCH_TIER_ENTERPRISE` and `SEARCH_ADD_ON_LLM` in `app.json` — the two lines that make generated answers possible, and the two that cost money — attached to `bookshelf-ds`. Google's documentation confirms the mapping: the Enterprise features toggle is the tier, the Generative Responses features toggle is the add-on. Then one query in the app's **Preview** page, to see that the index is ready. The app can be created while the store is still indexing.

### Steps

1. Read the create body the kit's `04-create-app.sh` sends. Two fields are yours: the tier and the add-on. The console also asks for a company name, which the API body does not carry.

**KIT FILE — LOCAL. THE SCRIPT REPLACES DATASTORE\_ID WITH THE VALUE IN LAB.ENV; HERE YOU TICK THE STORE**

```
$ cat app.json # kit file, for reading
{
  "displayName": "Bookshelf search",
  "dataStoreIds": ["DATASTORE_ID"],
  "solutionType": "SOLUTION_TYPE_SEARCH",
  "searchEngineConfig": {
    "searchTier": "SEARCH_TIER_ENTERPRISE",
    "searchAddOns": ["SEARCH_ADD_ON_LLM"]
  }
}
```

2. On the **AI Applications** page click **Create app**. On the **Create app** page, the documentation's card for a search app is **Site search with AI mode** — click its **Create**. The name says "site" but the documentation gives it as the search app type; if today's console shows a separate card for a general **Search** app over your own data stores, that card is the one.
3. Two toggles, both **on**: **Enterprise edition features** ("Decide if you want Enterprise features for this app" — `SEARCH_TIER_ENTERPRISE`) and **Generative Responses** ("generative responses with Generative Responses features" — `SEARCH_ADD_ON_LLM`). Both are charged for. Without the second there is no generated answer in Parts D-3, E and F.
4. **Your app name**: `bookshelf-app`. **External name of your company or organization**: `Meridian Financial`. **Location of your app**: `global (Global)`. As in Part B, watch for the **id** the console derives from the name and **write it down**: it is `APP_ID` in Part D-3's resource string and in Part F's `lab.env`. Click **Continue**.
5. On the **Data stores** page tick `bookshelf-ds` (the one store you created — do not create a new one here), then click **Create**.

**EXPECTED OUTPUT. THE APP LIST AFTER CREATE — THE SAME THREE THINGS 04-CREATE-APP.SH READS BACK: DATASTOREIDS, SEARCHTIER, SEARCHADDONS**

| Apps          |        |              |        |            |                      |
|---------------|--------|--------------|--------|------------|----------------------|
| Name          | Type   | Data stores  | Region | Enterprise | Generative Responses |
| bookshelf-app | Search | bookshelf-ds | global | on         | on                   |

6. Readiness, and only readiness. Open **bookshelf-app**, click **Preview** in its navigation menu, and type a question that is *\*not\** the day's question — "**How much is the missed payment fee on a personal loan?**" (golden G12–G28 are priya's product questions; any of them will do). Results with snippets mean the index has caught up with Part B; a generated answer above them means the LLM add-on is on. If there are no results, the store is still indexing: wait, and read on in the deck. Do not ask the dispute question here; Part D-1 asks it first, on purpose, without the answer.

**EXPECTED OUTPUT. GOOGLE'S DOCUMENTATION: EACH RESULT HAS A TITLE, A SNIPPET AND A URL; A GENERATED ANSWER "MIGHT ALSO APPEAR" WHEN GENERATIVE RESPONSES FEATURES ARE ON**

Preview — bookshelf-app

[ How much is the missed payment fee on a personal loan? ]

<a generated answer, if Generative Responses are on>

Personal Loan — Product Handbook

... missed payment fee ...

<further results: title, snippet, URL>

### ✓ Check yourself

The app exists with the **Enterprise** and **Generative Responses** features on, attached to `bookshelf-ds` and nothing else.

You know the app's **id** as the console shows it, and you have written it down.

You know where the **Preview** page is, and the readiness question returned results.

## Troubleshooting

| Symptom                                                   | Cause                                                                     | Fix                                                                                                                                                              |
|-----------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No card that fits a search app over a Cloud Storage store | The console's cards have been renamed                                     | The card you want is the one that ends with a <b>Data stores</b> page listing <code>bookshelf-ds</code>                                                          |
| One of the two toggles is missing                         | The page hides it until a step later, or the tier is preselected          | Look for it on the next page; after creation, check the app's configuration for both features; if either is off, delete the app and create it again with both on |
| <b>Preview</b> returns no results for minutes             | Part B's index (or the schema re-index) is not ready                      | Wait a few minutes and retry. Not an error                                                                                                                       |
| <b>Preview</b> shows results but no generated answer      | <b>Generative Responses</b> is off, or the preview is set to results only | Check the app's configuration; look for an answer/summary toggle in the preview                                                                                  |
| 403 or "permission" anywhere                              | APIs or role — see Part B                                                 | Part A step 4; check the role                                                                                                                                    |

## Part D-1 — The day's question, no filter

### Goal

Ask the data store one question from Cloud Shell and read what comes back: five chunks, each from a document. Two of the documents have the same title. Then ask the answer endpoint the same question and write down what it says.

### Before you start Part D — two ids into lab.env

The console gave your data store and your app an **id** that is not the name you typed. Both scripts in Part D read the ids from `lab.env`, so put them in now.

**Data store id:** AI Applications → **Data Stores** → `bookshelf-ds` → **Details** tab → the ID field.

**App id:** AI Applications → **Apps** → `bookshelf-app` → **App overview** → the ID field (it is also in the page's web address, after `/engines/`).

Then in Cloud Shell: `cd ~/regai-d1-gcp-kit, nano lab.env`, and set `DATASTORE_ID=` and `APP_ID=` to those two values. Save with `Ctrl+O`, `Enter`, `Ctrl+X`.

### KIT FILE — LOCAL. YOUR IDS WILL DIFFER; THE POINT IS THAT NEITHER IS JUST THE NAME

```
$ cd ~/regai-d1-gcp-kit
$ grep -E '^(PROJECT_ID|DATASTORE_ID|APP_ID)=' lab.env
PROJECT_ID=my-project
DATASTORE_ID=bookshelf-ds_1758000000000
APP_ID=bookshelf-app_1758000000000
```

### Steps

1. Run the search. One command, one question, no filter. The script prints the request it sends, then a table.

### EXPECTED OUTPUT. YOUR RANKS AND CHUNK IDS WILL DIFFER. LOOK FOR THE TWO DOCUMENT NAMES

```
$ bash scripts/search.sh "Which form must a cardholder sign before a dispute can go to chargeback?"
question: Which form must a cardholder sign before a dispute can go to chargeback?
filter: (none)
+ curl -sS -X POST "https://.../dataStores/<your id>/servingConfigs/default_search:search" ...
  body:
    { "query": "...", "pageSize": 10,
      "contentSearchSpec": { "searchResultMode": "CHUNKS" } }

totalSize: <n>
rank  document                chunk  content (first 90 chars)
1     POL-CR-014              c<n>   # Credit Card Dispute and Chargeback Handling Policy ##...
2     POL-CR-014-v2          c<n>   # Credit Card Dispute and Chargeback Handling Policy ##...
3     ...
```

2. Read the **document** column and write down the rank of every `POL-CR-014` row and every `POL-CR-014-v2` row. Two documents, one title.
3. Find out what each of the two documents says the form is. Three commands, in the kit folder.

### KIT FILE — LOCAL

```
$ grep -h '<title>' corpus-html/POL-CR-014.html corpus-html/POL-CR-014-v2.html
<title>Credit Card Dispute and Chargeback Handling Policy</title>
<title>Credit Card Dispute and Chargeback Handling Policy (superseded)</title>
$ grep -o 'form FRM-CD-[0-9]' corpus-html/POL-CR-014.html corpus-html/POL-CR-014-v2.html
corpus-html/POL-CR-014.html:form FRM-CD-7
corpus-html/POL-CR-014-v2.html:form FRM-CD-6
$ jq -r 'select(.id|startswith("POL-CR-014"))
| "\(.id)\t\(.structData.status)\t\(.structData.version)"' metadata.jsonl
POL-CR-014-v2    superseded    2.4
POL-CR-014      approved     3.2
```

So: `POL-CR-014-v2` is the old version, marked **superseded**, and it says the form is **FRM-CD-6**. `POL-CR-014` is the current version, **approved**, and it says **FRM-CD-7**. The search returned chunks from both. Nothing in the table told you which one is still the rule.

4. Ask the answer endpoint the same question with no filter. Write down the answer word for word, and which document it cites.

```
EXPECTED OUTPUT. READ WHICH FORM IT NAMES AND WHICH DOCUMENT THE CITATION POINTS AT
$ bash scripts/answer.sh --no-filter "Which form must a cardholder sign before a dispute
can go to chargeback?"
question: Which form must a cardholder sign before a dispute can go to chargeback?
filter: (none)
+ curl -sS -X POST "https://.../engines/<your app id>/servingConfigs/default_search:answer" ...

answer:
<the answer text – does it say FRM-CD-7, FRM-CD-6, or both?>

citations:
  [1] <document title> – <first 100 characters of the cited chunk>
grounding: <score>
skipped: none
```

**✓ Check yourself**

Your search table has rows from both **POL-CR-014** and **POL-CR-014-v2**, and you wrote down their ranks. You have seen that the old version says **FRM-CD-6** and the current one says **FRM-CD-7**. You wrote down the unfiltered answer and the document it cited.

**Troubleshooting**

| Symptom                                                | Cause                                                                                                      | Fix                                                                                     |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| HTTP 404 ... DataStore ... not found                   | DATASTORE_ID in <code>lab.env</code> is the name, not the id                                               | Read the id from the data store's <b>Details</b> tab and put it in <code>lab.env</code> |
| HTTP 404 from <code>answer.sh</code>                   | APP_ID in <code>lab.env</code> is the name, not the id — or the app was created less than five minutes ago | Read the id from <b>App overview</b> ; wait five minutes after creating the app         |
| <code>totalSize: 0</code>                              | Indexing has not finished                                                                                  | Wait a few minutes and rerun                                                            |
| No <b>POL-CR-014-v2</b> row in the table               | It ranked lower than tenth                                                                                 | <code>PAGE_SIZE=20 bash scripts/search.sh "..."</code>                                  |
| <code>answer.sh</code> — HTTP 400 mentioning the model | ANSWER_MODEL in <code>lab.env</code> is not a value the API accepts                                        | Set <code>ANSWER_MODEL=stable</code> and rerun                                          |

## Part D-2 — The same search, three filters

### Goal

Run the same search three more times, each time with a filter string as the second argument, and compare the tables with D-1's: which rows disappear each time.

### Steps

1. Read the three filter strings. They live in `callers.json`; one per caller.

#### KIT FILE — LOCAL (CALLERS.JSON)

```
priya    internal      status: ANY("approved") AND classification: ANY("internal")
daniel   confidential  status: ANY("approved") AND classification:
ANY("internal","confidential")
mlro     restricted    status: ANY("approved") AND classification:
ANY("internal","confidential","restricted")
```

2. Filter 1 — approved documents only. Mind the quotes: single quotes around the whole string, double quotes inside it.

#### EXPECTED OUTPUT. NO POL-CR-014-V2 ROW THIS TIME

```
$ bash scripts/search.sh "Which form must a cardholder sign before a dispute can go to
chargeback?" 'status: ANY("approved")'
question: Which form must a cardholder sign before a dispute can go to chargeback?
filter:   status: ANY("approved")
...
rank  document      chunk  content (first 90 chars)
1     POL-CR-014     c<n>   ...
2     POL-CR-014     c<n>   ...
3     PRC-CR-030     c<n>   ...
```

3. Filter 2 — daniel's string: approved, and internal or confidential.

#### EXPECTED OUTPUT. THE SAME ROWS AS FILTER 1 — THE DISPUTE POLICY IS CONFIDENTIAL, AND DANIEL MAY SEE IT

```
$ bash scripts/search.sh "Which form must a cardholder sign before a dispute can go to
chargeback?" \
'status: ANY("approved") AND classification: ANY("internal","confidential")'
filter:   status: ANY("approved") AND classification: ANY("internal","confidential")
...
rank  document      chunk  content (first 90 chars)
1     POL-CR-014     c<n>   ...
```

4. Filter 3 — priya's string: approved, and internal only. Which document is gone?

#### EXPECTED OUTPUT. NO POL-CR-014 ROW OF EITHER VERSION

```
$ bash scripts/search.sh "Which form must a cardholder sign before a dispute can go to
chargeback?" \
'status: ANY("approved") AND classification: ANY("internal")'
filter:   status: ANY("approved") AND classification: ANY("internal")
...
rank  document      chunk  content (first 90 chars)
1     PRC-CR-030     c<n>   ...
2     <another approved, internal document>
```

5. Write two lines in your notes: which rows filter 1 removed compared with D-1, and which rows filter 3 removed compared with filter 2. No table says "removed" — the comparison is the evidence.

**✓ Check yourself**

Filter 1 and filter 2: no POL-CR-014-v2. Filter 3: no POL-CR-014 at all.

You can say what each clause did: `status` removed the superseded version for everyone; `classification` removed the confidential policy for priya.

You can say what the filter is **not**: an access control. Anyone who can call the API can send any string, or none. Choosing the string from who the caller is — that is the control, and it lives in the code that calls the API.

**Troubleshooting**

| Symptom                                               | Cause                                                                                    | Fix                                                                                  |
|-------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| HTTP 400<br>INVALID_ARGUMENT<br>mentioning the filter | <code>status</code> or <code>classification</code> is not marked indexable in the schema | Part B's schema step was skipped or has not finished re-indexing; do it, wait, rerun |
| <code>filter: (none)</code> although you typed one    | The quotes were wrong and the shell dropped the second argument                          | Copy the command exactly as printed                                                  |
| priya's table is empty                                | Nothing approved and internal matched                                                    | Possible. Note it, and try <code>PAGE_SIZE=20</code>                                 |

## Part D-3 — The answer endpoint, as daniel and as priya

### Goal

Ask the app for an answer twice — once as daniel, once as priya. The script picks the caller's filter string for you. Read what each one gets: an answer with a citation, or a refusal.

### Steps

#### 1. Ask as daniel.

**EXPECTED OUTPUT. READ THE FORM IT NAMES AND THE DOCUMENT IT CITES**

```
$ bash scripts/answer.sh --user daniel "Which form must a cardholder sign before a dispute
can go to chargeback?"
question: Which form must a cardholder sign before a dispute can go to chargeback?
filter:   status: ANY("approved") AND classification: ANY("internal","confidential")
+ curl -sS -X POST "https://.../engines/<your app id>/servingConfigs/default_search:answer" ...

answer:
<the answer — expect FRM-CD-7>

citations:
[1] Credit Card Dispute and Chargeback Handling Policy — # Credit Card Dispute ... ## 6. ...
grounding: <score>
skipped: none
```

#### 2. Ask as priya.

**EXPECTED OUTPUT. EITHER A SKIP REASON LIKE THIS, OR AN ANSWER FROM A DOCUMENT SHE MAY SEE — WRITE DOWN WHICH**

```
$ bash scripts/answer.sh --user priya "Which form must a cardholder sign before a dispute
can go to chargeback?"
question: Which form must a cardholder sign before a dispute can go to chargeback?
filter:   status: ANY("approved") AND classification: ANY("internal")
...

answer:
(no answer text)

citations:
(none)
grounding: (not returned)
skipped: NO_RELEVANT_CONTENT
```

#### 3. Compare the two. Same question, same app, same minute. One caller got the current rule with a citation; the other got nothing from the dispute policy. The only difference between the two requests was the filter string.

**✓ Check yourself**

daniel: an answer naming **FRM-CD-7**, cited to **POL-CR-014**.

priya: a skip reason (or an answer that cites no version of **POL-CR-014**).

You can say which line of the request made the difference.

**Troubleshooting**

| Symptom                                      | Cause                                                                               | Fix                                                                                                     |
|----------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| HTTP 404                                     | APP_ID in <code>lab.env</code> is wrong, or the app is not ready yet                | Read the id from <b>App overview</b> ; wait five minutes after creating the app                         |
| HTTP 400 mentioning the model                | ANSWER_MODEL in <code>lab.env</code> is not a value the API accepts                 | Set ANSWER_MODEL= <code>stable</code> and rerun                                                         |
| HTTP 403 mentioning Enterprise or LLM add-on | <b>Generative Responses</b> or <b>Enterprise edition features</b> is off on the app | Turn both on in the app's settings; wait five minutes                                                   |
| daniel's answer names FRM-CD-6               | The filter did not remove the superseded document                                   | Run D-2 filter 1 again; if <b>POL-CR-014-v2</b> is still there, the schema step in Part B has not taken |

## Part E — The console preview pane

### Goal

No request body. Open the app in the console, find its **Preview** page, and ask the day's question. Read the answer with its citations highlighted; open one citation and read the chunk it points at. Then compare with Part D-3: the same endpoint, but the preview carries no filter and no preamble that you set — which makes it the unfiltered view unless it lets you set one.

### Steps

1. In the console, open **AI Applications**, click **bookshelf-app**, then **Preview** in its navigation menu — the page you used once in Part C for readiness.
2. Type the day's question — **"Which form must a cardholder sign before a dispute can go to chargeback?"** — and press Enter. Read the generated answer. Highlighted spans are citations: each one is a `citations[].startIndex/endIndex` from the response you read in Part D-3, drawn on the screen.
3. Click one citation. The panel that opens is `references[n].chunkInfo`: the chunk's content, with the ancestor headings at the top, and the document it came from. Confirm the document title — is it the approved policy, the superseded one, or does the preview show both?
4. Look for a filter control in the preview. If there is none, the preview is the **unfiltered** view — the same question Part D-1's last step asked with no `filter`. Compare the two answers. If there is one, set `priya`'s string and compare with Part D-3.
5. Ask one of the eight unanswerable questions from Appendix A — "What is the current rate on a two-year fixed mortgage?" (U01) is a content gap for every caller. Note how the preview shows a refusal, if it does.

### ✓ Check yourself

You found the preview without help, and you can name the page you were on.

You clicked a citation and read the chunk: headings first, then the passage — the same object D-3's `references[n].chunkInfo.content` held.

You know whether the preview applied a filter, and therefore whose view it was.

You can say what the preview is for: the app's owner looking at the app. A staff-facing tool is your code calling `:answer` with the caller's filter — which is what the D-3 body, and `evaluate.py`, model.

### Troubleshooting

| Symptom                                                | Cause                                                                          | Fix                                                                                               |
|--------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| No <b>Preview</b> where the manual says                | The console has been relabelled since these steps were written                 | Look in the app's navigation menu for the page with a search box; ask in the room if it has moved |
| The preview shows no generated answer, only results    | <b>Generative Responses</b> is off, or the preview mode is set to results only | Check the app's configuration; look for an answer/summary toggle in the preview                   |
| The preview's answer differs from daniel's in Part D-3 | No filter, no preamble, possibly a different model version                     | Expected — that is step 4                                                                         |
| The preview shows the superseded policy in a citation  | It is the unfiltered view                                                      | Correct behaviour for an unfiltered search; it is what Part D-2 exists to remove                  |

## Part F — Measuring: forty questions, the judge, the thresholds

### Goal

`evaluate.py` sends every golden question to `:answer` with its caller's filter, scores each response from the response alone (key-fact correctness, abstention, document-level recall@5 and MRR), sends the answered ones to the Vertex AI evaluation service for `groundedness` and `question_answering_quality`, prints a per-question table and a summary, checks `eval.yaml`, and exits 1 on any breach. Run it as shipped, then with `--no-status-filter` — which drops `status: ANY("approved")` from every caller's string and lets the superseded policy back in — and compare the two summaries.

### Why this part keeps one script

There is no console page that sends forty questions to the answer endpoint, each with its caller's filter and the Bookshelf preamble, and scores the replies against a golden set — so the loop is `evaluate.py`, the one script in this edition, run from Cloud Shell. The judge half of `evaluate.py` logs its runs to **Vertex AI Experiments**, which the console can show — step 7.

**Before you start:** `eval.yaml` ships with starting thresholds. Step 6 sets them from your own with-filter run; until then a `BELOW THRESHOLD` line on the with-filter run is a prompt to read the rows, not a verdict.

### Cloud Shell set-up for this part (once)

The kit is already in Cloud Shell from Part A, and `lab.env` already has your `PROJECT_ID`. Two more values must now match what the console created: `DATASTORE_ID` and `APP_ID` as the console shows them (Part B step 6, Part C step 4 — if the console added a suffix, the suffix goes here). `BUCKET` is not used by `evaluate.py`; leave the rest as it is.

**KIT FILE — LOCAL, WITH YOUR THREE IDS. EVALUATE.PY READS IT AND REFUSES TO RUN WHILE PROJECT\_ID IS EMPTY**

```
$ cd ~/regai-d1-gcp-kit
$ nano lab.env
$ grep -v '^#' lab.env | grep .
PROJECT_ID=my-project
LOCATION=global
BUCKET=${PROJECT_ID}-regai-corpus
BUCKET_LOCATION=europe-west2
DATASTORE_ID=bookshelf-ds
APP_ID=bookshelf-app
ANSWER_MODEL=stable
VERTEX_LOCATION=europe-west2
```

Then make the Python virtual environment `evaluate.py` runs in. It lives in `$HOME` so it survives Cloud Shell restarts; the `pip install` is the slow step, a minute or two.

**THE VENV. AFTER ANY CLOUD SHELL RESTART: SOURCE ~/REGAI-VENV/BIN/ACTIVATE**

```
$ python3 -m venv ~/regai-venv
$ ~/regai-venv/bin/pip install --quiet --upgrade pip
$ ~/regai-venv/bin/pip install --quiet "google-cloud-aiplatform[evaluation]" \
  pandas pyyaml requests
$ source ~/regai-venv/bin/activate
```

## Steps

1. Run the cheap check: three questions, no judge. This proves auth, `lab.env` and the loop work before you spend forty answer calls.

EXPECTED OUTPUT. THE HEADER LINES AND THE FILTER STRINGS ARE THE SCRIPT'S; THE PER-ROW VALUES ARE YOURS. THE REQUEST BODY IS THE ONE YOU PASTED IN PART D-3

```
$ python scripts/evaluate.py --limit 3 --skip-judge
== 3 questions -> :answer on app bookshelf-app in my-project (with-filter)
model stable; filters from callers.json
priya status: ANY("approved") AND classification: ANY("internal")
daniel status: ANY("approved") AND classification: ANY("internal","confidential")
mlro status: ANY("approved") AND classification: ANY("internal",...,"restricted")
first request body (the others differ only in query.text and filter):
{
  "query": { "text": "How long does a customer have to dispute a card transaction?" },
  "answerGenerationSpec": { ... },
  "searchSpec": { "searchParams": { "maxReturnResults": 5, "filter": "..." } }
}
[ 1/3] G01 daniel skipped=- refs=<n> <s>s
[ 2/3] G02 daniel skipped=- refs=<n> <s>s
[ 3/3] G03 daniel skipped=- refs=<n> <s>s

== Vertex AI evaluation: skipped (--skip-judge)

== per question (with-filter)
id caller expect abstain correct rec@5 mrr refs cited skipped
-----
G01 daniel answer no yes 1.00 1.00 <n> <n> -
G02 daniel answer no yes 1.00 1.00 <n> <n> -
G03 daniel answer no yes 1.00 1.00 <n> <n> -

== summary (with-filter)
n 3
...
```

2. The full run, with the judge, saving everything to a file. Forty `:answer` calls in sequence, then the evaluation service. **Start it, then read on** — the forty calls take minutes, and the judge takes minutes more.

EXPECTED OUTPUT — FIRST HALF: THE FORTY CALLS, THEN THE JUDGE. IF THE SERVICE'S KEYS ARE NOT GROUNDEDNESS/MEAN AND QUESTION\_ANSWERING\_QUALITY/MEAN THE SUMMARY SHOWS (NOT RUN)

```
$ python scripts/evaluate.py --out run-with-filter.json
== 40 questions -> :answer on app bookshelf-app in my-project (with-filter)
...
[ 1/40] G01 daniel skipped=- refs=<n> <s>s
...
[40/40] U08 daniel skipped=<reason> refs=0 <s>s

== Vertex AI evaluation on <n> answers (project my-project, location europe-west2)
summary_metrics from the service:
row_count: <n>
groundedness/mean: <x>
groundedness/std: <x>
question_answering_quality/mean: <x>
question_answering_quality/std: <x>
```

EXPECTED OUTPUT — SECOND HALF: THE TABLE, THE SUMMARY, THE THRESHOLDS. EVERY NUMBER IS YOURS; THE SKIPPED COLUMN IS SHORTENED HERE TO FIT THE PAGE

```
== per question (with-filter)
id   caller  expect  abstain correct  rec@5   mrr refs cited ground  qaq  skipped
-----
G01  daniel  answer      no    yes   1.00   1.00   <n>   <n>   <x>   <x>   -
...
U01  priya   abstain     yes    no    -      -      0     0     -     -     NO_RELEVANT_...
...

== summary (with-filter)
n                40
n_answerable     32
n_unanswerable   8
recall_at_5      0.<n>
mrr              0.<n>
correctness      0.<n>
correct_abstain_rate  0.<n>
false_abstain_rate  0.<n>
false_answer_rate  0.<n>
zero_citation_rate  0.<n>
grounding_score_mean  0.<n>
groundedness     <x>
question_answering_quality <x>

== thresholds (eval.yaml)
ok                recall_at_5                = 0.<n>   (min <t>)
...

wrote run-with-filter.json
all thresholds met — exit 0
```

3. Read the table before the summary. Find the rows where `abstain` and `expect` disagree — an answerable question refused (`abstain yes` on a `G` row) or an unanswerable one answered (`abstain no` on a `U` row) — and any row with `cited 0` on an answer. Those rows are what the rates in the summary are made of.

4. Now the weakened run. `--no-status-filter` removes `status: ANY("approved")` from all three strings; the classification clause stays. The superseded dispute policy is back in scope for daniel's questions.

**EXPECTED OUTPUT. WHICH METRIC MOVES — CORRECTNESS, FALSE\_ANSWER\_RATE, GROUNDEDNESS — AND WHETHER THE RUN BREACHES IS THE POINT OF THE PART**

```
$ python scripts/evaluate.py --no-status-filter --out run-no-status.json
== 40 questions -> :answer on app bookshelf-app in my-project (no-status-filter)
  model stable; filters from callers.json
  priya   classification: ANY("internal")
  daniel  classification: ANY("internal","confidential")
  mlro    classification: ANY("internal","confidential","restricted")
...
== summary (no-status-filter)
...
correctness           0.<n>
false_answer_rate     0.<n>
groundedness          <x>
...

== thresholds (eval.yaml)
...
BELOW THRESHOLD <metric>                = 0.<n>  (<min|max> <t>)
...

<n> threshold breach(es) — exit 1
```

5. Put the two summaries side by side and fill in the table. Then answer: which number moved, and does that match where the superseded chunks could reach (daniel's dispute questions, and nobody else's)?

| metric                     | with filter | --no-status-filter | what it counts           |
|----------------------------|-------------|--------------------|--------------------------|
| recall_at_5                |             |                    | document level, of 32    |
| mrr                        |             |                    | document level, of 32    |
| correctness                |             |                    | key facts present, of 32 |
| correct_abstain_rate       |             |                    | of the 8                 |
| false_abstain_rate         |             |                    | of the 32                |
| false_answer_rate          |             |                    | of the 8                 |
| zero_citation_rate         |             |                    | of answers given         |
| groundedness               |             |                    | the judge                |
| question_answering_quality |             |                    | the judge                |
| exit code                  |             |                    | 0 or 1                   |

If `groundedness` did not move, that is not reassuring: the judge grades the answer against the chunks the model was shown, and an answer taken from the superseded chunk is grounded in that chunk. The metrics that know what *should* have happened are `correctness` and the abstention rates. If none of them moved either, note it, and look at which of daniel's questions the superseded chunks reached in `run-no-status.json`.

- Set the thresholds from what you measured. With the with-filter summary in hand, edit `eval.yaml`: each "must not get worse" threshold a little below what the run showed, each "must never happen" threshold at its ceiling. Then rerun both commands and check: with-filter exits 0, weakened exits 1 with at least one `BELOW THRESHOLD` line. That pair — a run that passes and a weakened run that fails — is the measurement.

**EXAMPLE — THE OUTPUT OF FIXTURE MODE (`--from-fixtures`, NO API CALL) ON THE KIT'S EXAMPLE DATA, WITH THE SHIPPED THRESHOLDS. IT SHOWS EVERY LINE THE THRESHOLD CHECK CAN PRINT; THE VALUES ARE THE EXAMPLE'S, NOT THE SERVICE'S**

```
$ python scripts/evaluate.py --from-fixtures | tail -14
== thresholds (eval.yaml)
ok          recall_at_5          = 0.969 (min 0.8)
ok          mrr                  = 0.932 (min 0.7)
ok          correctness          = 0.938 (min 0.7)
ok          correct_abstain_rate = 0.875 (min 0.75)
ok          false_abstain_rate   = 0.031 (max 0.15)
BELOW THRESHOLD false_answer_rate = 0.125 (max 0.1)
BELOW THRESHOLD zero_citation_rate = 0.031 (max 0.0)
SKIPPED     groundedness        (no value this run)
SKIPPED     question_answering_quality (no value this run)

2 threshold breach(es) – exit 1
```

- Optional — the judge's runs in the console. `evaluate.py` names each evaluation run's experiment `bookshelf-d1-with-filter` and `bookshelf-d1-no-status-filter`; Google's own material confirms that an `EvalTask` with an `experiment` name logs its runs to **Vertex AI Experiments**. Open **Vertex AI → Experiments**, find the two experiments, and read the same `groundedness` and `question_answering_quality` means the script printed.

### ✓ Check yourself

`--limit 3 --skip-judge` printed three rows and a summary with `n 3`.

The full run printed 40 rows, a `summary_metrics` from the `service:` block with two `/mean` keys, and a summary where `groundedness` and `question_answering_quality` are numbers, not (`not run`).

You can name the rows where `abstain` and `expect` disagreed, and say what each did against the golden set.

The weakened run's summary differs from the with-filter one, and you can say which line moved and why — or, if none did, you looked at `run-no-status.json` to see which daniel questions the superseded chunks reached.

With the thresholds you set in step 6: with-filter exits 0, `--no-status-filter` exits 1 with at least one `BELOW THRESHOLD` line.

## Troubleshooting

| Symptom                                                                                | Cause                                                                                                                    | Fix                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ModuleNotFoundError: No module named 'requests'</code> (or <code>pandas</code> ) | The <code>venv</code> is not active                                                                                      | <code>source ~/regai-venv/bin/activate</code>                                                                                                                                                                                                                           |
| <code>PROJECT_ID</code> is empty:<br><code>copy lab.env.example ...</code>             | <code>lab.env</code> not filled in                                                                                       | See the set-up at the top of this part                                                                                                                                                                                                                                  |
| HTTP 404 on every question                                                             | <code>DATASTORE_ID</code> or <code>APP_ID</code> in <code>lab.env</code> is not the id the console generated             | Copy the ids from the console (Part B step 6, Part C step 4), suffix included                                                                                                                                                                                           |
| HTTP 400 from <code>:answer -- ... model ...</code>                                    | The <code>modelVersion</code> string was rejected                                                                        | <code>ANSWER_MODEL=stable</code> in <code>lab.env</code> ; the script reads it                                                                                                                                                                                          |
| <code>!! vertexai.evaluation could not be imported ...</code>                          | The evaluation SDK is missing or has moved                                                                               | <code>pip install --upgrade "google-cloud-aiplatform[evaluation]"</code> in the <code>venv</code> . If the import is gone for good, the newer GenAI-client evaluation path replaces <code>EvalTask</code> . <code>--skip-judge</code> gives the local metrics meanwhile |
| 403 from the evaluation service                                                        | The Agent Platform API ( <code>aiplatform.googleapis.com</code> ) is not enabled, or you lack Vertex AI User             | Part A step 4; check the role                                                                                                                                                                                                                                           |
| <code>groundedness</code> (not run) with no import error                               | The service's <code>summary_metrics</code> keys are not <code>groundedness/mean / question_answering_quality/mean</code> | Read the printed <code>summary_metrics</code> from the service: block; the key match is in <code>run_judge()</code>                                                                                                                                                     |
| Judge scores look like 0-1, not 1-5                                                    | The scale differs from what <code>eval.yaml</code> assumes                                                               | Re-base the two judge thresholds; note it                                                                                                                                                                                                                               |
| <code>BELOW THRESHOLD</code> on the with-filter run                                    | <code>eval.yaml</code> still has the shipped values, or the run really is below                                          | Read the rows the rate is made of; step 6 sets the thresholds from your run                                                                                                                                                                                             |
| The run is slow                                                                        | 40 answer calls in sequence, then the judge                                                                              | Expected. Read on                                                                                                                                                                                                                                                       |

## Teardown — at the end of the day, in every project

The app carries the Enterprise tier and the LLM add-on and is billed while it exists. Delete in this order — the app first (it must go before the data store), then the data store, then the bucket; it is what `99-teardown.sh` does. The two Vertex AI experiments cost nothing to keep.

1. **AI Applications → Apps → `bookshelf-app` → Delete** (a delete control on the app's page or in its row menu). Confirm. Wait until the app is gone from the list.
2. **AI Applications → Data Stores → `bookshelf-ds` → Delete**. Confirm. A data store that will not delete usually still has the app attached — check step 1 finished.
3. **Cloud Storage → Buckets → tick `<project-id>-regai-corpus` → Delete**. The dialog asks you to type a confirmation; the objects go with the bucket.

### Keep these four files

`callers.json` (the filter strings), `golden.json` (the forty questions), `eval.yaml` (the thresholds you set) and the preamble in `answer.sh / evaluate.py`. They are what made the service answer correctly, to the right person, and prove it — and they are yours whatever retriever is behind them next time.

## Appendix A — The golden set (golden.json)

40 real-style staff questions over the 13-document Bookshelf: 32 with a reference answer and the **key facts** a correct answer must contain, and 8 that must be refused (the last column gives the reason: `not_in_corpus`, or `not_retrieved` — the answer exists in a document that caller may not see). Each question names its caller; `evaluate.py` sends that caller's filter with it.

`relevant_chunk_ids` in the file (not shown) were labelled against a hand-rolled chunking and do not match the service's chunk ids (`c0`, `c5` ...). `evaluate.py` uses only the document part of each id — `POL-CR-014#1` → `POL-CR-014` — which is why `recall@5` and `MRR` are at document level.

### One golden item, in full

```
{
  "id": "G03",
  "question": "Which form must a cardholder sign before a dispute can go to chargeback?",
  "caller": "daniel",
  "relevant_chunk_ids": [
    "POL-CR-014#4",
    "POL-CR-014#5"
  ],
  "reference_answer": "The Cardholder Dispute Declaration, form FRM-CD-7, signed by the cardholder (electronic signature in the app is acceptable); a dispute cannot progress to chargeback without it (POL-CR-014, 6.1).",
  "key_facts": [
    "FRM-CD-7"
  ],
  "expected": "answer",
  "tags": [
    "exact-code",
    "planted-fact",
    "multi-chunk"
  ]
}
```

| id  | caller | expected | question                                                                         | key facts              |
|-----|--------|----------|----------------------------------------------------------------------------------|------------------------|
| G01 | daniel | answer   | How long does a customer have to dispute a card transaction?                     | 120 days               |
| G02 | daniel | answer   | What does POL-CR-014 say about the chargeback window?                            | 120 days               |
| G03 | daniel | answer   | Which form must a cardholder sign before a dispute can go to chargeback?         | FRM-CD-7               |
| G04 | daniel | answer   | Within how many business days must the Bank acknowledge a dispute in writing?    | 2 business days        |
| G05 | daniel | answer   | For a subscription payment, from which date does the dispute window run?         | most recent            |
| G06 | daniel | answer   | Who can approve an exception to the 120-day dispute window?                      | Senior Dispute Officer |
| G07 | daniel | answer   | How long are credit card dispute records kept after a case is closed?            | 7 years                |
| G08 | daniel | answer   | When must provisional credit be applied for an unauthorised transaction dispute? | 10 business days       |
| G09 | daniel | answer   | What is the maximum a teller may hold in the till during the day?                | £15,000                |
| G10 | daniel | answer   | Which cash withdrawals need 24 hours' notice from the customer?                  | £10,000                |
| G11 | daniel | answer   | What happens if a till is £30 short at the end of the day?                       | recount                |
| G12 | priya  | answer   | What is the representative APR on a personal loan of £10,000 over 3 years?       | 6.9%                   |

|     |        |         |                                                                                      |                 |
|-----|--------|---------|--------------------------------------------------------------------------------------|-----------------|
| G13 | priya  | answer  | Can a customer take a payment holiday on their personal loan?                        | 2 months        |
| G14 | priya  | answer  | How much is the missed payment fee on a personal loan?                               | £12             |
| G15 | priya  | answer  | How long does a customer have to change their mind after signing a loan agreement?   | 14 days         |
| G16 | priya  | answer  | What interest rate does the Everyday Saver pay on a balance of £20,000?              | 3.40%           |
| G17 | priya  | answer  | Is there a daily limit on withdrawals from an Everyday Saver to an external account? | £25,000         |
| G18 | priya  | answer  | How much of an Everyday Saver balance is protected by the FSCS?                      | £85,000         |
| G19 | priya  | answer  | How much notice do customers get before a savings rate goes down?                    | 14 days         |
| G20 | priya  | answer  | Which documents does a customer need to prove their address?                         | List B          |
| G21 | priya  | answer  | How often are high-risk customers re-verified?                                       | 12 months       |
| G22 | priya  | answer  | How long can a cardholder freeze a mislaid card in the app?                          | 7 days          |
| G23 | priya  | answer  | Is there a fee for an emergency card replacement by courier?                         | £15             |
| G24 | priya  | answer  | What is the nightly hotel rate cap in London?                                        | £150            |
| G25 | priya  | answer  | How much mileage can I claim for using my own car on business?                       | 45 pence        |
| G26 | priya  | answer  | What is the maximum period a privileged access grant can last?                       | 90 days         |
| G27 | priya  | answer  | How often must service account credentials be rotated?                               | 180 days        |
| G28 | priya  | answer  | How long are complaint files retained?                                               | 5 years         |
| G29 | mlro   | answer  | What must a colleague not do after submitting an internal SAR?                       | tipping off     |
| G30 | mlro   | answer  | How quickly must the MLRO review an internal suspicious activity report?             | 2 business days |
| G31 | mlro   | answer  | What is the fixed return on the 5-year Buy Local Bonds series?                       | 4.75%           |
| G32 | mlro   | answer  | What reason code do uncleared colleagues see on an account restricted after a SAR?   | REV-01          |
| U01 | priya  | abstain | What is the current rate on a two-year fixed mortgage?                               | not_in_corpus   |
| U02 | priya  | abstain | Can a customer open a Junior ISA for their child?                                    | not_in_corpus   |
| U03 | priya  | abstain | What are the Saturday opening hours of the Manchester branch?                        | not_in_corpus   |
| U04 | daniel | abstain | What is the employer pension contribution rate for staff?                            | not_in_corpus   |
| U05 | priya  | abstain | How long does a customer have to dispute a card transaction?                         | not_retrieved   |
| U06 | priya  | abstain | What is the maximum cash a teller can hold in the till?                              | not_retrieved   |
| U07 | daniel | abstain | What must a colleague not do after filing an internal SAR?                           | not_retrieved   |
| U08 | daniel | abstain | What is the minimum investment in Buy Local Bonds?                                   | not_retrieved   |

## Appendix B — The callers and their filter strings (callers.json)

Three callers, three clearances, three strings. In Part D you paste the string into the request body — `filter` at the top level of a search body, `searchSpec.searchParams.filter` in an answer body — and `evaluate.py` sends it on every request. The `status` clause is on all three; the `classification` list grows with clearance. `department` is indexable but no string uses it.

| id     | Name and role                       | Department                 | Clearance    |
|--------|-------------------------------------|----------------------------|--------------|
| priya  | Priya Nair — Customer Contact agent | Customer Contact           | internal     |
| daniel | Daniel Okafor — Dispute Officer     | Cards Operations           | confidential |
| mlro   | Sarah Whitmore — Deputy MLRO        | Financial Crime Compliance | restricted   |

### THE THREE STRINGS, VERBATIM

```
priya  status: ANY("approved") AND classification: ANY("internal")
daniel status: ANY("approved") AND classification: ANY("internal","confidential")
mlro   status: ANY("approved") AND classification: ANY("internal","confidential","restricted")
```

### THE SAME THREE, AS THE JSON LINE YOU PASTE INTO A REQUEST BODY (INNER QUOTES ESCAPED)

```
priya
  "filter": "status: ANY(\"approved\") AND classification: ANY(\"internal\")"
daniel
  "filter": "status: ANY(\"approved\") AND classification: ANY(\"internal\",\"confidential\")"
mlro
  "filter": "status: ANY(\"approved\") AND classification: ANY(\"internal\",\"confidential\",\"restricted\")"
```

The file's own note: "One caller per clearance level. 'filter' is the exact Discovery Engine filter string `answer.sh` -- user NAME and `evaluate.py` send. The filter is a metadata filter on indexable fields, not an access control: anyone who can call the API can send a different filter."

The grammar the kit uses: `field: ANY("v1", "v2")`, clauses joined with `AND`, on fields the schema marked **Indexable**. If a filter is rejected with `INVALID_ARGUMENT`, the likely cause is the schema edit not saved, or its re-index not finished, before the search.

## Appendix C — eval.yaml, verbatim

### The values are starting points

The file below is what ships. Part F step 6 sets every value from your own with-filter summary — a little below what you saw for "must not get worse" metrics, at the ceiling for "must never happen" ones — and checks that the weakened run then breaches at least one.

#### EVAL.YAML, AS SHIPPED — VERBATIM, AT A SMALLER SIZE SO NO LINE WRAPS

```
# Thresholds read by scripts/evaluate.py.
#
# The values below are starting points. Set them from your own numbers:
# 1. run python scripts/evaluate.py once, with the status filter on, and read the summary,
# 2. put each "must not get worse" threshold a little below what you saw,
# 3. put each "must never happen" threshold at its ceiling,
# 4. run python scripts/evaluate.py --no-status-filter and check that the
#    weakened run breaches at least one of them (that is the Part F point).
#
# {min: x} -> the value must be >= x; {max: x} -> the value must be <= x.
# A metric with no value in a run (for example the judge metrics in fixture mode)
# is printed as SKIPPED and does not fail the run.
# Any breach prints a BELOW THRESHOLD line and makes evaluate.py exit 1.

k: 5

thresholds:
  # --- computed locally from the answer API's references and answerText ---
  recall_at_5:      {min: 0.80} # doc-level: the answering document is among the references
  mrr:              {min: 0.70} # ...and near the top of them
  correctness:      {min: 0.70} # answerable questions whose answer contains every key fact
  correct_abstain_rate: {min: 0.75} # of the 8 unanswerable, at least 6 abstain
  false_abstain_rate: {max: 0.15} # of the 32 answerable, at most 4 wrongly abstain
  false_answer_rate:  {max: 0.10} # of the 8 unanswerable, none should get an answer; 0.10 allows none
  zero_citation_rate: {max: 0.00} # an answer with no citation must never reach a caller

  # --- from the Vertex AI evaluation service (EvalTask), 1-5 scale as the SDK's ---
  # --- pointwise templates score; check the scale in your first run and re-base ---
  groundedness:      {min: 4.0}
  question_answering_quality: {min: 3.5}
```